Thermodynamics II

Gas Turbine Thermodynamic Analysis

Project One

Pouya Asghari

Student ID: 810600037

School of Mechanical Engineering
College of Engineering, University of Tehran
First semester, academic year 1403–1404 (Iranian calendar)

About this English edition

This edition translates the supplied Persian report and retains its equations, screenshots, and code excerpts. Section titles have been added for navigation. Source code printed from the Word report is a documentary transcription, including source spacing; use the native MATLAB files for execution. A separate editorial appendix identifies discrepancies without silently correcting the historical model. No MATLAB or App Designer calculation was executed during conversion. The screenshot shows an interface, not a verified numerical result.

Contents

1 Compressor inlet and compression **2**

2 Combustion and product composition **2**

2.1 Carbon dioxide dissociation 3

2.2 Nitric oxide formation 3

3 Adiabatic combustion and excess-air search **4**

4 Turbine inlet properties: state 4 **5**

5 Turbine outlet properties: state 5 **6**

6 Exergy losses and efficiencies **7**

7 MATLAB App Designer interface **7**

A Editorial review and reproduction limits **9**

1 Compressor inlet and compression

First, the compressor inlet temperature and pressure are used with the CoolProp library to determine enthalpy and entropy. Since the pressure ratio is known, the pressures at states 2, 3, and 4 can also be determined; the combustion chamber operates at constant pressure.

We first assume an ideal compressor with isentropic compression. This gives s_{2s} and hence h_{2s}. The ideal compressor work is obtained from the enthalpy difference, and the compressor efficiency then gives the actual enthalpy at state 2. The source refers to the difference $h_1 - h_{2s}$ in its prose; the code below uses the positive compressor-work convention $h_{2s} - h_1$.

```
% State 2
P(2)=P(1)*rp;
s2s=s(1);
h2s=CoolProp.PropsSI('H','S',s2s,'P',P(2),'Air');
w_c=(h2s-h(1))/eta_c;
h(2)=h(1)+w_c;
T(2)=CoolProp.PropsSI('T','H',h(2),'P',P(2),'Air');
s(2)=CoolProp.PropsSI('S','H',h(2),'P',P(2),'Air');
```

Figure 1: Original compressor calculation screenshot.

2 Combustion and product composition

The calculation so far is the straightforward part. If heat were added without mixing air and fuel, the enthalpy and entropy at states 4 and 5 could be obtained directly, after which the remaining quantities would follow. Here, however, a hydrocarbon fuel C_xH_y mixes with air containing 21% oxygen and 79% nitrogen. Each chemical species must therefore be considered separately.

$$C_xH_y + \left(x + \frac{y}{4}\right)(1 + EA)(O_2 + 3.76N_2) \longrightarrow bCO_2 + cH_2O + dCO + eNO + fO_2 + gN_2. \quad (1)$$

As indicated by this reaction, seven unknowns must be determined: EA, b, c, d, e, f, g. Here EA represents excess air relative to the stoichiometric amount. The equations in the supplied implementation specialize the general hydrocarbon reaction to methane.

The mass balances give four equations. In the original displayed equations these are written as

$$\text{C balance:} \quad 1 = b + d, \quad (2)$$

$$\text{H balance:} \quad 4 = 2c, \quad (3)$$

$$\text{O balance:} \quad 4(1 + EA) = 2b + 2 + d + e + 2f, \quad (4)$$

$$\text{N balance:} \quad 4(1 + EA)3.76 = e + 2g. \quad (5)$$

The oxygen balance above uses $c = 2$; the code retains c symbolically in that position.

There are six product species and four elements, so two equilibrium-constant equations can be written: $6 - 4 = 2$.

2.1 Carbon dioxide dissociation

The first reaction is

$$CO_2 \longrightarrow CO + \tfrac{1}{2}O_2. \tag{6}$$

The Gibbs relations are then used to calculate K_p.

```
% CO2 ---> CO + 1/2 O2

% Ru=8.314;

hfO2_1 = 0;
hfCO2 = -393520; %J/mol
hfCO = -110530;  %J/mol

hO2_1 = CoolProp.PropsSI('Hmolar','T',T(4),'P',0.5*P(2),'O2');
hCO = CoolProp.PropsSI('Hmolar','T',T(4),'P',P(2),'CO');
hCO2 = CoolProp.PropsSI('Hmolar','T',T(4),'P',P(2),'CO2');

h0O2_1 = CoolProp.PropsSI('Hmolar','T',T0,'P',0.5*P0,'O2');
h0CO = CoolProp.PropsSI('Hmolar','T',T0,'P',P0,'CO');
h0CO2 = CoolProp.PropsSI('Hmolar','T',T0,'P',P0,'CO2');

sO2_1 = CoolProp.PropsSI('Smolar','T',T(4),'P',0.5*P(2),'O2');
sCO= CoolProp.PropsSI('Smolar','T',T(4),'P',P(2),'CO');
sCO2= CoolProp.PropsSI('Smolar','T',T(4),'P',P(2),'CO2');

gO2_1 = hfO2_1+hO2_1-h0O2_1-T(4)*sO2_1;
gCO2 = hfCO2+hCO2-h0CO2-T(4)*sCO2;
gCO = hfCO+hCO-h0CO-T(4)*sCO;

delta_G1 = gCO + 0.5*gO2_1 - gCO2;

kp1=exp(-delta_G1/(Ru*T(4)));
```

Figure 2: Original code for the first equilibrium constant.

2.2 Nitric oxide formation

The same procedure is applied to the second reaction:

$$\tfrac{1}{2}N_2 + \tfrac{1}{2}O_2 \longrightarrow NO. \tag{7}$$

```
% 1/2 N2 + 1/2 O2 ---> NO
Ru=8.314;
cpNO=1.05;
molmass_NO = 30;
hfO2_2 = 0;
hfN2 = 0; %J/mol
hfNO = 90250;  %J/mol ---------------------------------------- "https://chem.libretexts.org/"

hO2_2 = CoolProp.PropsSI('Hmolar','T',T(4),'P',0.5*P(2),'O2');
hN2 = CoolProp.PropsSI('Hmolar','T',T(4),'P',0.5*P(2),'N2');
% hNO = CoolProp.PropsSI('Hmolar','T',T(4),'P',P0,'NO');

h0O2_2 = CoolProp.PropsSI('Hmolar','T',T0,'P',0.5*P0,'O2');
h0N2 = CoolProp.PropsSI('Hmolar','T',T0,'P',0.5*P0,'N2');
% h0NO = CoolProp.PropsSI('Hmolar','T',T0,'P',P0,'NO');

sO2_2 = CoolProp.PropsSI('Smolar','T',T(4),'P',0.5*P(2),'O2');
sN2= CoolProp.PropsSI('Smolar','T',T(4),'P',0.5*P(2),'N2');
% sNO= CoolProp.PropsSI('Smolar','T',T(4),'P',P0,'NO');

hrNO = cpNO*molmass_NO*(T(4)-T0);
s0NO=210.76;      %J/mol-K-----------------------------------------from "webbook.nist.gov"
sNO=s0NO-Ru*log(1);
gO2_2 = hfO2_2+hO2_2-h0O2_2-T(4)*sO2_2;
gN2 = hfN2+hN2-h0N2-T(4)*sN2;
gNO = hfNO+hrNO-T(4)*sNO;

delta_G2 = gNO - 0.5*gN2 - 0.5*gO2_2;

kp2=exp(-delta_G2/(Ru*T(4)));
```

Figure 3: Original code for the second equilibrium constant.

The source states that NO properties are unavailable in the CoolProp library used for this project. NO is omitted where its influence is assumed negligible, while approximate relations are used to include it later.

3 Adiabatic combustion and excess-air search

Four mass-balance equations and two equilibrium-constant equations are now available. One more equation is needed to determine the remaining unknown, excess air.

The combustion chamber is assumed adiabatic, with no heat loss. The total energy of the incoming fuel and air should therefore be approximately equal to that of the products. To impose this condition, a `for` loop assigns EA values from 0 to 2 in increments of 0.1. In each iteration, the enthalpies of the fuel, air, and products are calculated and their difference is updated. The intention is to bring the energy equation close to balance and find a physically representative excess-air value.

For a prescribed EA in each iteration, the problem becomes six equations in six unknowns. These are solved using `vpasolve`. The product molar coefficients are calculated in each loop, after which the remaining energy relations can be evaluated.

```
% C.C (find EA)

P( 3)= P(2);
P( 4)= P(3);
yrN2=0.79;
yrO2=0.21;
yrCH4 = 1;
```

```
deltaH =10^8;

%  CH 4 + 2(1+EA)(O2+3.76N2) ----> b CO2 + c H2O + d CO + e NO + f O2 + g N2

s4=0;
HProd =0;
HFuel =1;
HAir =1;
for  EA=0:0.1:2
     syms   b c d e f g ;
     eqns =[1 ==  b+d  , 4 == 2*c , 4*(1+EA) == 2*b+c+d+e+2*f , 4*3.76*(1+EA) == e+2*g , kp1 == ((d*f^0.5)/
     b)*((P(2)/P0)/( b+c+d+e+f+g ))^0.5 , kp2 == e/((g^0.5)*(f^0.5))*((P(2)/P0)/( b+c+d+e+f+g ))^0];
    var=[ b,c ,d,e,f,g ];
    [ b,c ,d,e,f,g ]= vpasolve ( eqns,var );
     yp = b+c+d+e+f+g ;

    ypCO2=double(b/ yp );
    ypH2O=double(c/ yp );
     ypCO =double(d/ yp );
     ypNO =double(e/ yp );
    ypO2=double(f/ yp );
    ypN2=double(g/ yp );

    hOCO2 =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'CO2' );
    hOH2O =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'H2O' );
    hOCO =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'CO' );
     % hONO =  CoolProp.PropsSI ('Hmolar','T ',T 0,'P',ypNO*P0,'NO');
    hOO2_p =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'O2' );
    hON2_p =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'N2' );

    hpCO2=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypCO2*P(3), 'CO2' )-hOCO2;
    hpH2O=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypH2O*P(3), 'H2O' )-hOH2O;
     hpCO =  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' , ypCO *P(3), 'CO' )-hOCO;
     hpNO =  cpNO * molmass_NO *(T(4)-T0);
    hpO2_p=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypO2*P(3), 'O2' )-hOO2_p;
    hpN2_p=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypN2*P(3), 'N2' )-hON2_p;
    hOCH4=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'Methane' );
    hpCH4=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,P(2), 'Methane' )-hOCH4;

    hOO2_r=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'O2' );
    hpO2_r=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,0.21*P(2), 'O2' )-hOO2_r;

    hON2_r=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'N2' );
    hpN2_r=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,0.79*P(2), 'N2' )-hON2_r;

     HFuel  = 1*(hfCH4+hpCH4);
     HAir  = 2*(1+ EA)* hpO2_r + 2*3.76*(1+EA)*hpN2_r;
     HProd  = double(b)*(hfCO2+hpCO2) + double(c)*(hfH2O+hpH2O) + double(d)*( hfCO+hpCO ) + double(e)*(
     hfNO+hpNO ) + double(f)*(hpO2_p) + double(g)*(hpN2_p);
     q_in =abs( abs( HProd )-abs( HAir )-abs( HFuel ));
     if ( q_in <= deltaH )
         DHfinal = q_in ;
         EAfinal =EA;
     end
     deltaH = q_in ;
end
EA= EAfinal ;
```

4 Turbine inlet properties: state 4

After determining EA, the corresponding product coefficients can be calculated. To obtain the enthalpy and entropy at state 4, immediately before the turbine, a relation of the following form is written for each species:

$$h = h_f + h_T - h_0. \tag{8}$$

The following code is transcribed from the report. It explicitly assigns $EA = 1.4$ after the search.

```
% State 4

Ru=8.314;
EA=1.4;
syms   b c d e f g ;
eqns =[1 ==  b+d  , 4 == 2*c , 4*(1+EA) == 2*b+c+d+e+2*f , 4*3.76*(1+EA) == e+2*g , kp1 == ((d*f^0.5)/b)*((
    P(2)/P0)/( b+c+d+e+f+g ))^0.5 , kp2 == e/((g^0.5)*(f^0.5))*((P(2)/P0)/( b+c+d+e+f+g ))^0];
var=[ b,c ,d,e,f,g ];
[ b,c ,d,e,f,g ]= vpasolve ( eqns,var );
yp = b+c+d+e+f+g ;

ypCO2=double(b/ yp );
ypH2O=double(c/ yp );
ypCO =double(d/ yp );
ypNO =double(e/ yp );
ypO2=double(f/ yp );
ypN2=double(g/ yp );


h0CO2 =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'CO2' );
h0H2O =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'H2O' );
h0CO =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'CO' );
% h0NO =  CoolProp.PropsSI ('Hmolar','T ',T 0,'P',P0,'NO');
h0O2_p =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'O2' );
h0N2_p =  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'N2' );


hpCO2=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypCO2*P(3), 'CO2' )-h0CO2;
hpH2O=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypH2O*P(3), 'H2O' )-h0H2O;
hpCO =  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' , ypCO *P(3), 'CO' )-h0CO;
hpNO =  cpNO * molmass_NO *(T(4)-T0);
hpO2_p=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypO2*P(3), 'O2' )-h0O2_p;
hpN2_p=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (4), 'P' ,ypN2*P(3), 'N2' )-h0N2_p;

h0CH4=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'Methane' );
hpCH4=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,P(2), 'Methane' )-h0CH4;

h0O2_r=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'O2' );
hpO2_r=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,0.21*P(2), 'O2' )-h0O2_r;

h0N2_r=  CoolProp.PropsSI ( 'Hmolar' , 'T ' ,T 0, 'P' ,P0, 'N2' );
hpN2_r=  CoolProp.PropsSI ( ' Hmolar ' , 'T ' ,T (2), 'P' ,0.79*P(2), 'N2' )-h0N2_r;

HFuel  = 1*(hfCH4+hpCH4);
HAir  = 2*(1+ EA)* hpO2_r + 2*3.76*(1+EA)*hpN2_r;
HProd  = double(b)*(hfCO2+hpCO2) + double(c)*(hfH2O+hpH2O) + double(d)*( hfCO+hpCO ) + double(e)*( hfNO+
    hpNO ) + double(f)*(hpO2_p) + double(g)*(hpN2_p);
MolProd =44+18+28+30+32+28;
h( 4)= abs( HProd )*1000/ MolProd ;

spCO2= CoolProp.PropsSI ( 'S' , 'T ' ,T (4), 'P' ,P(4), 'CO2' );
spH2O= CoolProp.PropsSI ( 'S' , 'T ' ,T (4), 'P' ,P(4), 'H2O' );
spCO = CoolProp.PropsSI ( 'S' , 'T ' ,T (4), 'P' ,P(4), 'CO' );
spO2= CoolProp.PropsSI ( 'S' , 'T ' ,T (4), 'P' ,P(4), 'O2' );
spN2= CoolProp.PropsSI ( 'S' , 'T ' ,T (4), 'P' ,P(4), 'N2' );

sp =ypCO2*spCO2 + ypH2O*spH2O +  ypCO * spCO  + ypO2*spO2 + ypN2*spN2;

s( 4)= sp ;
```

5 Turbine outlet properties: state 5

Once the enthalpy and entropy at state 4 are known, the turbine efficiency can be used to determine the corresponding outlet quantities at state 5. The source writes

$$s_{5s} = s_4, \qquad P_5 = P_1, \tag{9}$$

$$h_{5s} = h_4 - w_t/\eta_t, \qquad h_5 = h_4 - w_t. \tag{10}$$

To find the temperature at state 5, and consequently its entropy, the source again uses a search intended to bring the governing relations into agreement.

```
% State 5(after gas turbine)
s5s= s( 4);
P( 5)= P(1);

h5s= h( 4)- w_t / eta_t ;
h( 5)= h(4)- w_t ;
T5LB = 650;
deltaS  = 1e10;

for  T5s = (300):1:(1000)
    s5sCO2 =  CoolProp.PropsSI ( 'Smolar' , 'T ' ,T 5s, 'P' ,ypCO2*P(5), 'CO2' );
    s5sH2O =  CoolProp.PropsSI ( 'Smolar' , 'T ' ,T 5s, 'P' ,ypH2O*P(5), 'H2O' );
    s5sCO =  CoolProp.PropsSI ( 'Smolar' , 'T ' ,T 5s, 'P' ,ypCO*P(5), 'CO' );
    s5sO2 =  CoolProp.PropsSI ( 'Smolar' , 'T ' ,T 5s, 'P' ,ypO2*P(5), 'O2' );
    s5sN2 =  CoolProp.PropsSI ( 'Smolar' , 'T ' ,T 5s, 'P' ,ypN2*P(5), 'N2' );

    s5sTest0 = s5sCO2*ypCO2 + s5sH2O*ypH2O + s5sCO* ypCO  + s5sO2*ypO2 + s5sN2*ypN2;
    s5sTest= s5sTest0/(( MolProd )   /1000);
     if  (abs(s5sTest-s5 s)< deltaS )
        T5sf=T5s;
        s5sf=s5sTest;
     end
     deltaS  = abs(s5sTest-s5s);
end
s( 5)=s5sTest;
T( 5)=T5sf;
```

6 Exergy losses and efficiencies

Finally, the source calculates exergies, losses, and efficiencies using the following expressions.

```
% Exergy Analysis

ex_comp =T0*(s(2)- s( 1));
ex_cc =T0*(s(4)-s(2)- q_in / T( 2));
ex_turb =T0*(s(5)- s( 4));
ex_cycle = ex_turb+ex_cc+ex_comp ;

% Efficiency
                                                    q_out =(h(5)- h( 1));
eta_cycle =1-q_out/ q_in ;

eta2_comp=1-ex_comp/ w_c ;
eta2_cc=1-ex_cc/( q_in );
eta2_turb=1-ex_turb/ w_t ;
```

These expressions are reproduced as submitted. Their interpretation and numerical consistency require the checks in Appendix A.

7 MATLAB App Designer interface

The calculation is converted into a function and inserted into App Designer:

```
function [P,T,h,s,ex_comp,ex_cc,ex_turb,ex_cycle,eta_cycle,eta2_comp,eta2_cc,eta2_turb,EA,b,c,d,e,f,g,AF] =
      GasTur(T1,P1,rp,eta_c,eta_t,T4,w_t)
```

The application created in App Designer is shown below.

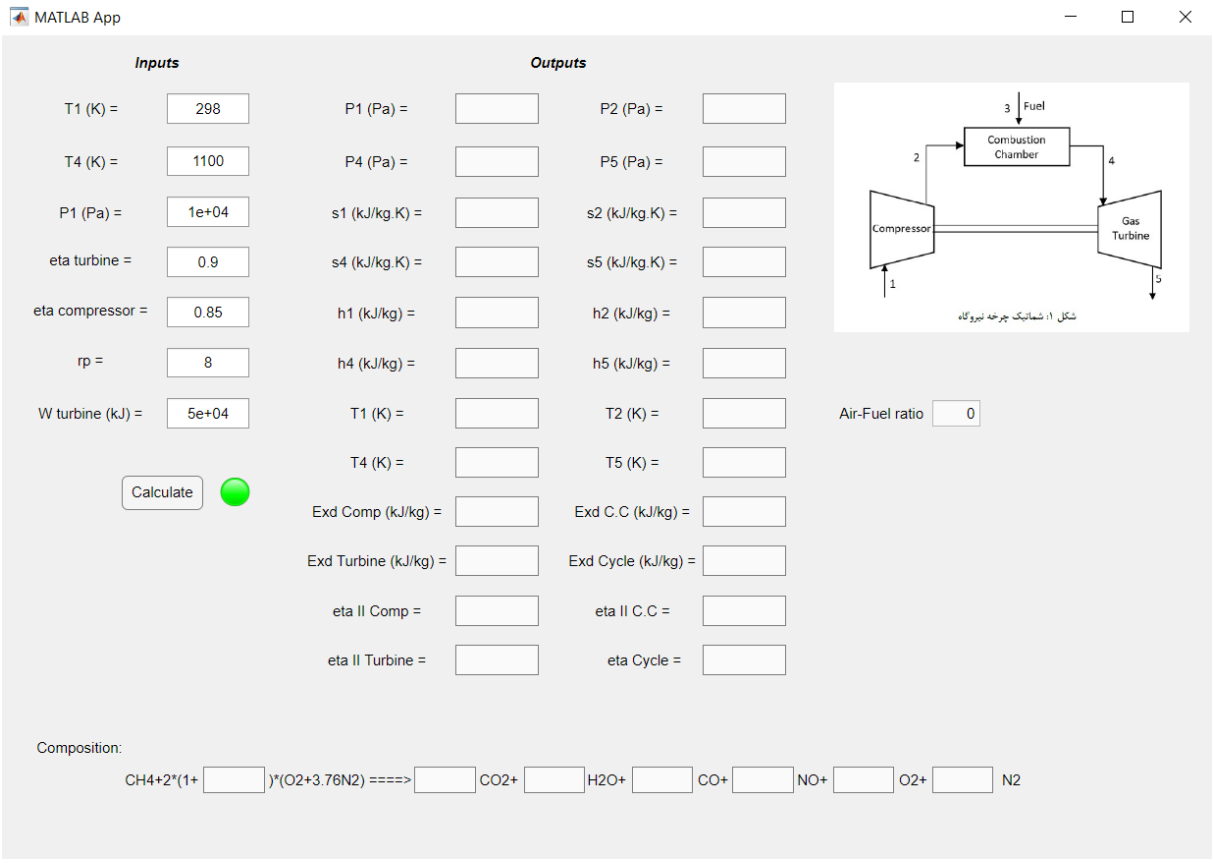


Figure 4: Original MATLAB application interface.

The application uses callbacks to display composition, pressures, entropies, enthalpies, temperatures, exergy quantities, efficiencies, and the air–fuel ratio:

```
%Composition
            app.EA.Value = string( EA);
            app. b.Value =string(double(b));
            app. c.Value =string(double(c));
            app. d.Value =string(double(d));
            app. e.Value =string(double(e));
            app. f.Value =string(double(f));
            app. g.Value =string(double(g));
            %Show pressures
            app.P 1PaEditField_2.Value=string(P(1));
            app.P2PaEditField.Value =string(P(2));
            app.P4PaEditField.Value =string(P(4));
            app.P5PaEditField.Value =string(P(5));
            %Show entropies
            app.s1kJkgKEditField.Value =string(s(1)/1000);
            app.s2kJkgKEditField.Value =string(s(2)/1000);
            app.s4kJkgKEditField.Value =string(s(4)/1000);
            app.s5kJkgKEditField.Value =string(s(5)/1000);
            %Show enthalpies
            app.h1kJkgEditField.Value =string(h(1)/1000);
            app.h2kJkgEditField.Value =string(h(2)/1000);
            app.h4kJkgEditField.Value =string(h(4)/1000);
            app.h5kJkgEditField.Value =string(h(5)/1000);
            %Show temperatures
            app.T 1KEditField_2.Value=string(T(1));
            app.T2KEditField.Value =string(T(2));
            app.T 4KEditField_2.Value=string(T(4));
            app.T5KEditField.Value =string(T(5));
            %Show Exergies
            app.ExdCompkJkgEditField.Value =string( ex_comp );
            app.ExdCCkJkgEditField.Value =string( ex_cc );
            app.ExdTurbinekJkgEditField.Value =string( ex_turb );
            app.ExdCyclekJkgEditField.Value =string( ex_comp+ex_turb+ex_cc );
            %Others
            app.etaCycleEditField.Value =string( eta_cycle );
            app.etaIICompEditField.Value =string(eta2_comp);
            app.etaIICCEditField.Value =string(eta2_cc);
            app.etaIITurbineEditField.Value =string(eta2_turb);
            app.AirFuelratioEditField.Value =AF;
```

A Editorial review and reproduction limits

The following observations were added during conversion; they are not translated claims of the original author.

1. The report, Live Script, and application contain different numerical expressions. For example, the report prints `q_out=h(5)-h(1)`, while the Live Script divides this difference by 9. The Live Script later divides `ex_cc` by 900; the app includes divisions by 100 in component expressions. These versions must be reconciled before comparing efficiencies.

2. The excess-air search compares each residual to the preceding iteration rather than a retained global minimum. It does not guarantee the best candidate. It is followed by `EA=1.4`, overriding the selected value. The final `q_in` is not explicitly recomputed with that fixed composition.

3. The report uses `MolProd/1000` in the outlet entropy conversion; the native code uses `(MolProd-30)*2/10000`. Neither expression is a documented, composition-dependent mean molar mass. The product mass per mole of fuel should be based on the solved coefficients before converting a reaction-total molar quantity to a mass basis.

4. The source combines mass-specific pure-species entropy with mole fractions at state 4. Its enthalpy and entropy mixture bases, partial-pressure treatment, and NO approximation need a consistent formulation. Chemical and physical exergy contributions need explicit definitions and reference states before the combustion exergy expression is treated as validated.

5. `s(5)` is assigned the last trial entropy, rather than the entropy stored with the selected trial temperature. The selected temperature is sought through the isentropic condition even though state 5 is labeled as the actual outlet. Consistency with actual outlet enthalpy and turbine efficiency remains to be checked.

6. The enthalpy of formation assigned to water should be checked against the intended product phase and reference state. Absolute values in the energy residual also require review; they do not substitute for a consistent signed energy balance. No corrected numerical answer is supplied here.

7. Native code contains an absolute Windows path. The included `GasTur.m` removes the Live Script wrapper and changes only that path to a repository-relative one. Its equations remain historical and unverified. The native app is unchanged and still needs its embedded path edited.

8. The archive supplies a legacy CoolProp SWIG wrapper and a Windows 64-bit MEX binary. MATLAB, Symbolic Math Toolbox (for `syms` and `vpasolve`), and a compatible CoolProp interface are required. Native MATLAB execution and binary compatibility were not tested.

The source contains no completed numerical results table or validation study. The app screenshot has empty output fields. This repository documents a coursework model and interface; it does not establish verified efficiency, emissions, or plant performance.